

VIT BHOPAL UNIVERSITY

VITyarthi – Your Learning Destination

PROJECT INFORMATION

Project title	Student grade calculator
Course title	Introduction To Problem Solving And programming
Course code	CSE 1021
Course type	Flipped course
Course credits	4
Professor	Dr. Vikas panti
Slot	A21+A22+A23+B14+D21+D22
Submitted by	Aman Raj
Registration number	26BAI10661
College	VIT Bhopal University
Program	B.Tech CSE AI/ML
Year / Semester	1st Year, 1st Semester
Submission date	30/09/26

INTRODUCTION :

Every school and college has to do the same job after an examination: record the marks of each student, work out the percentage, decide the grade and be able to show the result whenever it is asked for. When this is done by hand with a calculator and a register, mistakes in addition, division or grade assignment are common and the process is slow.

This project is a small-scale simulation of such a result-processing desk, written in Python with a menu-driven console interface. The user adds a student with the marks of five subjects, and the program calculates the percentage and grade automatically. Stored records can be displayed, searched by roll number, or shown as a short result card. Control returns to the menu after each operation.

The application is intentionally lightweight. It uses only core Python features (functions, dictionaries, lists, loops and conditional statements) and needs no external library, which keeps it easy to install, run, review and grade as a first-year academic project.

Key characteristics of the system

- Menu-driven interaction
- Dictionary-based student record and list-based storage
- Automatic percentage and grade calculation
- Function-per-operation code structure
- Clear messages for no records, unknown roll number or an invalid choice

PROBLEM STATEMENT :

Manual result processing is slow and error-prone. Marks may be added incorrectly, percentages rounded inconsistently and grades assigned by different rules for different students. A small structured program can remove these problems by keeping the grade rules in one place and applying the same rules to every student.

The proposed system accepts a student's name, roll number and five marks, calculates the percentage, assigns a grade, and stores the record. It can then display all records, search by roll number and show an individual result. The project focuses on a clearly defined academic scope rather than attempting to reproduce a full examination management system: the goal is correctness and clarity of implementation rather than breadth of features.

PROJECT OBJECTIVES :

The main objective is to develop an understandable Python program that models a basic student grade calculation workflow. Additional objectives are to apply programming concepts in a complete project workflow, from requirements through design, implementation and testing.

- Apply core Python concepts: functions, dictionaries, lists, loops and conditionals.
- Store each student as a dictionary with name, roll number, marks, percentage and grade.
- Read the marks of five subjects with a loop and calculate the percentage as their average.
- Assign a grade of A+, A, B, C, D or F according to fixed percentage slabs.
- Display all records, search by roll number and show an individual result.
- Provide a simple menu that does not crash on an invalid menu choice.
- Test the program with a documented sequence of inputs and record the results.

Together, these objectives ensure that the finished project is not just a working script, but a small, well-organised piece of software that follows recognisable engineering practice.

SCOPE OF THE PROJECT :

The current project covers a single-user, single-session grade calculation simulation through a command-line interface. It focuses on entering student marks, computing percentage and grade, and retrieving stored results, using only standard Python features.

Editing or deleting records, subject names, rank lists, permanent storage, login and graphical interfaces are outside the current scope. They are listed under future enhancements because they need infrastructure beyond what a single-semester course project is expected to deliver.

- In scope: add student, display all students, search by roll number, show result
- In scope: percentage calculation and grade slabs
- In scope: terminal interface
- Out of scope: persistent storage (files or database)
- Out of scope: editing, deleting, login, rank list
- Out of scope: GUI or web interface

FUNCTIONAL REQUIREMENTS :

The functional requirements define the operations available to the user through the terminal menu.

- Accept a student name, roll number and marks of five subjects.
- Calculate the percentage as the average of the five marks.
- Assign a grade: A+ (90 and above), A (80), B (70), C (60), D (50), F (below 50).
- Store each student as a record and confirm with 'Student added successfully!'.
- Display name, roll number, marks, percentage and grade of every stored student.
- Report 'No student records found.' when no student has been added.
- Search a student by roll number and show the full details, or report 'Student not found.'.
- Show a result card (name, roll number, percentage, grade) for a roll number.
- Handle invalid menu choices without terminating the program.
- Exit cleanly with a thank-you message.

NON-FUNCTIONAL REQUIREMENTS :

Non-functional requirements describe qualities expected from the system beyond the specific operations it performs.

- Usability: simple numbered menu and clear prompts.
- Accuracy: percentage and grade are calculated by the program, never typed by the user.
- Maintainability: one function per operation, so each can be read and changed separately.
- Performance: instant response for the small list of records held in memory.
- Resource efficiency: no external libraries required.
- Portability: runs on any system with a standard Python 3 interpreter.
- Testability: operations can be checked repeatedly with a fixed input sequence.

REQUIREMENTS TRACEABILITY MATRIX :

The table below maps each functional requirement to the function primarily responsible for implementing it, and to the test case that verifies it. This traceability view helps confirm that every requirement is both implemented and tested.

Req. ID	Requirement	Responsible function	Verified by
FR-1	Accept name, roll number and five marks	add_student()	Add student test
FR-2	Calculate percentage	calculate_grade()	Grade tests
FR-3	Assign grade by slab	calculate_grade()	Grade A+ to F and boundary tests
FR-4	Store record and confirm	add_student()	Add student test
FR-5	Display all students	display_students()	Display students test
FR-6	Report no records	display_students()	Display empty test
FR-7	Search by roll number	search_student()	Search found / not found tests
FR-8	Show result card	show_result()	Show result tests
FR-9	Handle invalid menu choice	Main menu loop	Invalid menu choice test
FR-10	Exit with thank-you message	Main menu loop	Exit test

SYSTEM ARCHITECTURE :

The application follows a simple layered idea. The main menu loop receives the user's choice and calls the matching operation function. The operation functions read from and write to the shared students list, and print their result back to the console.

Keeping each operation in its own function separates user interaction from business rules to a useful degree. For example, the grade slab logic lives only inside `calculate_grade()`, so it can be changed in one place without touching the menu or the `add_student()` function.

- User to Main menu loop
- Main menu loop to Operation function (add / display / search / result)
- `add_student()` to `calculate_grade()` to students list
- Search and result functions read the students list
- Result message to Console to User

USE CASE ANALYSIS :

The main actor is the Teacher or the Student (or an exam-desk operator working for them). The actor interacts with the system through the terminal menu, and each menu option represents a use case. Every use case begins at the main menu and returns to it once the operation is complete.

- Add Student
- Display All Students
- Search Student
- Show Result
- Exit

DETAILED WORKFLOW :

When the program starts, an empty students list is created and the menu is displayed inside a while True loop. The selected operation is processed and the result is displayed. The loop continues until the user selects the exit option.

For adding a student, the system reads the name and roll number, then reads five marks in a for loop and stores them in a list. `calculate_grade()` finds the percentage as sum of marks divided by the number of marks and chooses the grade with an if / elif chain. A dictionary holding all details is appended to the students list.

- Start
- Create empty students list
- Display menu
- Read choice
- Process request
- Display result
- Return to menu
- Exit on option 5

SEQUENCE OF OPERATIONS :

A typical add-student interaction begins with the user choosing option 1. The program reads the name and roll number, reads five marks, calls `calculate_grade()`, builds the record and appends it to the list.

- User to Menu (choice 1)
- Menu to `add_student()`
- `add_student()` reads name, roll number and five marks
- `add_student()` to `calculate_grade(marks)` returns (percentage, grade)
- `add_student()` builds the student dictionary
- `add_student()` appends it to the students list
- Result to Console to User

MODULE AND COMPONENT DESIGN :

The program is divided into focused functions so that each has a clear, single responsibility. This makes the code easier to navigate, test and extend. The menu is numbered as follows: 1 Add student, 2 Display all students, 3 Search student, 4 Show result, 5 Exit.

Function	Purpose
calculate_grade(marks)	Takes the list of marks, returns the percentage and the grade.
add_student()	Reads name, roll number and five marks, builds the record and appends it to the list.
display_students()	Lists every stored student, or reports that no records exist.
search_student()	Reads a roll number and prints full details of the matching student, or a not-found message.
show_result()	Reads a roll number and prints a short result card, or a not-found message.
Main menu loop	Displays the menu, reads the choice and calls the matching function.

Each function has a small, clear purpose, which keeps the code short and makes it straightforward to locate the logic responsible for any menu option.

DATA MODEL AND STORAGE DESIGN :

All records are held in a global list named students. Each student is a dictionary with five keys: name, roll, marks, percentage and grade. The marks are kept as a list of five floating-point numbers. A list of dictionaries was chosen because it preserves the order in which students were added and lets every field be read by name.

```
students = []          # after adding Aman Raj, roll 101:
# students = [{
#     "name": "Aman Raj",
#     "roll": "101",
#     "marks": [95.0, 92.0, 88.0, 90.0, 95.0],
#     "percentage": 92.0,
#     "grade": "A+"
# }]
```

The roll number is stored as text (string), so it is compared with the text typed during search. The data lives only in memory while the program runs, so it is lost when the program closes. Adding file or database storage is listed under future enhancements.

INPUT VALIDATION AND ERROR HANDLING :

The current version keeps validation deliberately simple. The rules that it enforces are listed below.

- Viewing records when none exist shows 'No student records found.'
- Searching or showing a result for an unknown roll number shows 'Student not found.'
- Invalid menu choices show an error and the menu is displayed again.
- Percentage and grade are always calculated by the program, never typed by the user.
- Menu choice is read as text, so a wrong menu entry can never crash the program.

The current version does not use try/except blocks, so non-numeric input for a mark is not handled gracefully and stops the program with a ValueError. Marks are also not checked to lie between 0 and 100, and duplicate roll numbers are not rejected. These are recorded honestly as limitations and planned as future enhancements.

CORE ALGORITHM LOGIC :

This section describes, at a conceptual level, the logic behind the most important operations. The notation is illustrative pseudocode rather than literal source code.

Calculate grade

```
function calculate_grade(marks):
    percentage = sum(marks) / number of marks
    if percentage >= 90: grade = "A+"
    else if percentage >= 80: grade = "A"
    else if percentage >= 70: grade = "B"
    else if percentage >= 60: grade = "C"
    else if percentage >= 50: grade = "D"
    else: grade = "F"
    return percentage, grade
```

Add student

```
function add_student():
    read name, roll_no
    marks = empty list
    repeat 5 times:
        read mark as number, append to marks
    percentage, grade = calculate_grade(marks)
    append {name, roll, marks, percentage, grade} to students
```

Search student

```
function search_student():
    read roll_no
    for each student in students:
        if student.roll == roll_no:
            print details and return
    print "Student not found."
```

PROCESSING AND GRADING LOGIC :

The percentage is the average of the five subject marks, which is equal to the percentage when each subject is marked out of 100. The grade is then chosen from the percentage using the following slabs:

Percentage	Grade
90 or more	A+
80 to below 90	A
70 to below 80	B
60 to below 70	C
50 to below 60	D
Below 50	F

A worked example for each grade, using the same inputs as the test table, is shown below.

Marks entered	Total	Percentage	Grade
95, 92, 88, 90, 95	460	92.0	A+
80, 80, 80, 80, 80	400	80.0	A
70, 70, 70, 70, 70	350	70.0	B
60, 60, 60, 60, 60	300	60.0	C
50, 50, 50, 50, 50	250	50.0	D
40, 45, 30, 50, 35	200	40.0	F

The important point is that each slab uses 'greater than or equal to' on its lower limit, so a percentage of exactly 90.0 receives A+ and exactly 80.0 receives A. The if / elif chain is checked from the highest slab downwards, so the first matching slab decides the grade.

IMPLEMENTATION AND PYTHON CONCEPTS :

The project applies functions, dictionaries, lists, loops, conditional statements and console input/output. These concepts work together to create a complete application rather than isolated code examples.

- Dictionary for each student record
- List of dictionaries for all students
- List for the five marks
- Functions for each operation, with a return of two values (percentage, grade)
- while True loop for the menu
- for loop with range(5) to read marks, and for loops to display and search
- if / elif / else for menu selection and grade slabs
- Type conversion with float() for marks
- Built-in functions sum(), len() and round()
- String concatenation with str() for the prompt 'Subject 1', 'Subject 2', ...
- Console input and output

Using these fundamentals together, rather than in isolation, was a deliberate choice: it mirrors how introductory concepts are combined in real, working software.

TESTING STRATEGY AND TEST CASES :

The program was run with a sequence of inputs covering the normal path, each grade slab and the error messages. The outputs were compared with hand-calculated values, and the results matched in every case.

Test	Input	Expected result	Status
Display, no records	Option 2 on a new session	No student records found.	Pass
Search, no records	Option 3, roll 101 on a new session	Student not found.	Pass
Add student	Option 1, Aman Raj, 101, marks 95, 92, 88, 90, 95	Student added successfully!	Pass
Display students	Option 2 after adding	Percentage 92.0 %, Grade A+	Pass
Search found	Option 3, roll 101	Student Found! with full details	Pass
Search not found	Option 3, roll 999	Student not found.	Pass
Show result	Option 4, roll 101	Result card: 92.0 %, A+	Pass
Show result, unknown	Option 4, roll 999	Student not found.	Pass
Grade A	Marks 80 x 5	80.0 %, Grade A	Pass
Grade B	Marks 70 x 5	70.0 %, Grade B	Pass
Grade C	Marks 60 x 5	60.0 %, Grade C	Pass
Grade D	Marks 50 x 5	50.0 %, Grade D	Pass
Grade F	Marks 40, 45, 30, 50, 35	40.0 %, Grade F	Pass
Boundary, exactly 90	Marks 89, 90, 91, 88, 92	90.0 %, Grade A+	Pass
Just below 90	Marks 89.9 x 5	89.9 %, Grade A	Pass
Invalid menu choice	Option 9	Invalid choice. Please try again.	Pass
Exit	Option 5	Thank you!, program ends	Pass

Cases known to fail (recorded honestly as limitations, not counted above): entering text such as abc for a mark stops the program with a ValueError, and marks outside the range 0 to 100 are accepted. For example, marks -10, 150, 0, 0, 0 give a percentage of 28.0 and Grade F.

SAMPLE OUTPUT :

The following console output was produced by running student_grade_calculator.py with the inputs listed in the test table.

Main menu

```
=====
STUDENT GRADE MANAGEMENT
=====
1. Add Student
2. Display All Students
3. Search Student
4. Show Result
5. Exit
=====
```

Adding a student

```
Enter your choice: 1
Enter student name: Aman Raj
Enter roll number: 101
Enter marks of 5 subjects:
Subject 1: 95
Subject 2: 92
Subject 3: 88
Subject 4: 90
Subject 5: 95
Student added successfully!
```

Display all students

```
----- STUDENT RECORDS -----

Name: Aman Raj
Roll No: 101
Marks: [95.0, 92.0, 88.0, 90.0, 95.0]
Percentage: 92.0 %
Grade: A+
```

Search student

```
Enter roll number to search: 101

Student Found!
Name: Aman Raj
Roll No: 101
Marks: [95.0, 92.0, 88.0, 90.0, 95.0]
Percentage: 92.0 %
Grade: A+
```

Show result

```
Enter roll number: 101

===== RESULT =====
Name       : Aman Raj
Roll No    : 101
Percentage : 92.0 %
Grade      : A+
=====
```

Error messages

```
Enter your choice: 9
Invalid choice. Please try again.

Enter your choice: 2          (no students added yet)
No student records found.
```

```
Enter roll number to search: 999
Student not found.
```

DESIGN DECISIONS :

The implementation choices were made to keep the project understandable while demonstrating clear organisation.

- One function per operation keeps responsibilities separate.
- A separate `calculate_grade()` function returns both percentage and grade, so the grading rules live in one place.
- Each student is a dictionary, so fields are read by name (`student['grade']`) instead of by position.
- The percentage and grade are stored at the time of adding, so display and search do not recalculate them.
- Menu choice is read as text and compared with strings, so a wrong menu entry can never crash the program.
- A while True menu loop keeps the program running until the user chooses to exit.
- A console interface avoids unnecessary external dependencies.

ASSUMPTIONS AND CONSTRAINTS :

A few simplifying assumptions were made to keep the project within the scope of a first-semester course, while still producing a usable tool.

- A single user runs the application on a single machine at a time.
- Every subject is marked out of 100, so the average of five marks is the percentage.
- Every student has exactly five subjects.
- The user enters marks as numbers and roll numbers as text.
- Roll numbers are assumed to be unique.
- The program is run from a terminal with a standard Python 3 interpreter available.
- Data only needs to exist for the length of one session.

CHALLENGES AND SOLUTIONS :

Getting the grade slabs right was the main challenge. Because a student can score exactly on a boundary such as 90 or 80, the comparison had to use 'greater than or equal to' and the checks had to run from the highest slab downwards. The solution was an if / elif chain that tests 90 first, then 80, and so on, so the first true condition decides the grade.

Storing a student in a form that is easy to print was solved by keeping each record as a dictionary, so display, search and result functions simply read the required keys.

Keeping the program readable as features were added was handled by giving each operation its own function instead of writing everything inside the menu loop.

PROJECT TIMELINE AND WORK BREAKDOWN :

The project was carried out in stages, moving from requirements to a working, tested application. The approximate breakdown of effort is shown below.

Stage	Activity
1	Requirements gathering and problem definition
2	Data structure and function design
3	Core implementation (calculate_grade, add student, display)
4	Search and result features
5	Menu integration and manual testing
6	Test execution and result recording
7	Report documentation

LEARNINGS AND OUTCOMES :

The project provides practical experience in converting a problem into a structured Python application and connecting user input, processing and output.

- Functions and program structure
- Dictionaries and lists, including a list of dictionaries
- Loops and conditional logic
- Percentage and grade calculation in code
- Manual test design
- Project documentation

LIMITATIONS AND FUTURE ENHANCEMENTS :

Limitations

- Data is not persistent; it is lost when the program closes.
- No exception handling: a non-numeric mark stops the program.
- Marks are not range-checked, so negative marks or marks above 100 are accepted.
- Blank names and blank roll numbers are accepted, and duplicate roll numbers are not rejected.
- Roll number search is an exact, case-sensitive match and returns only the first matching student.
- Records cannot be edited or deleted.
- Every student must have exactly five subjects, and subjects have no names.
- The percentage is a simple average, so it assumes each subject is out of 100.
- Floating-point numbers are used for marks, so marks are displayed like 95.0.

Future enhancements

- Add try/except blocks and reject marks outside 0 to 100.
- Reject duplicate and blank roll numbers.
- Add options to edit and delete a student record.
- Save records to a file (text, JSON or CSV) and load them on start.
- Add subject names, a variable number of subjects and pass/fail status.
- Add class average, topper and rank list.
- Use classes (OOP) for the student record.
- Build a GUI (Tkinter) or web interface.

CONCLUSION :

The Student Grade Calculator demonstrates how an everyday problem can be addressed with a structured Python application. It combines menu-driven input handling, dictionary-based student records, a list-based store and automatic percentage and grade calculation to implement the core steps of a result desk.

The current version provides a clear foundation that can be extended with input validation, persistence and a richer interface, while remaining suitable for an introductory academic project.

GLOSSARY OF TERMS :

Term	Meaning
Roll number	The identifier typed for a student and used to search the record.
Marks	The list of five numbers entered for the subjects of a student.
Percentage	The average of the five marks, i.e. sum of marks divided by 5.
Grade	A letter (A+, A, B, C, D or F) decided by the percentage slab.
Student record	A dictionary holding name, roll, marks, percentage and grade.
Dictionary	A Python data structure that stores key-value pairs.
List	An ordered Python collection, used for marks and for all students.
CLI	Command-Line Interface; a text-based way of using a program.

USER MANUAL – HOW TO RUN THE APPLICATION :

The following steps describe how to run the Student Grade Calculator from a terminal.

1. Install Python 3 if it is not already available on the system.
2. Open a terminal and navigate to the project folder.
3. Run the application with: `python student_grade_calculator.py`
4. Choose an option from the numbered menu (1 to 5).
5. To add a student: choose option 1, enter the name, the roll number and the five marks.
6. Choose option 2 to see all students, option 3 to search and option 4 to see a result by roll number.
7. Select option 5 to exit the program.

REFERENCES AND SUBMISSION CHECKLIST :

The implementation uses core Python concepts and course learning context. Before submission, all project details and runtime behaviour should be verified.

- Python documentation: docs.python.org
- CSE 1021 course learning materials
- Student-created implementation and test cases
- Run every menu option
- Verify student details and date
- Check the final ZIP contents